

Leia esse tutorial <https://pt-br.react.dev/learn/tutorial-tic-tac-toe> e, com base nele, faça um tutorial de como construir um jogo igual ao Termo (<https://term.ooo/>) usando React no Windows para alguém que nunca usou React e sabe bem pouco de JavaScript.

A ideia é seguir a mesma filosofia do tutorial da React: aprender construindo algo real, começando simples e evoluindo passo a passo. No tutorial oficial do jogo da velha, você aprende os conceitos centrais do React (componentes, props e state) criando um tabuleiro interativo. Nós vamos aplicar exatamente esses conceitos para criar uma versão simplificada do Termo.

Tutorial: Criando um clone simples do Termo com React

O que vamos construir

Um jogo que:

- Escolhe uma palavra secreta de 5 letras.
- Permite ao jogador tentar adivinhar.
- Mostra:
 -  letra certa no lugar certo
 -  letra existe mas está em outro lugar
 -  letra não existe
- Permite até 6 tentativas.

Exemplo:

Palavra secreta: CASAL

Tentativa: CALOR

Resultado:

    

Parte 1 — Preparando o ambiente no Windows

Instalar Node.js

Baixe o instalador em:

[Node.js](#)

Depois da instalação, abra o Prompt de Comando e execute:

```
node -v  
npm -v
```

Se aparecerem versões, está funcionando.

Instalar VS Code

Baixe:

[Visual Studio Code](#)

Criar o projeto React

Abra o terminal:

```
npm create vite@latest termo-react -- --template react
```

Entre na pasta:

```
cd termo-react
```

Instale dependências:

```
npm install
```

Execute:

```
npm run dev
```

O terminal mostrará algo parecido com:

```
http://localhost:5173
```

Abra esse endereço no navegador.

Parte 2 — Entendendo a estrutura


```
    </div>
  );
}
```

Você já tem o esqueleto do jogo.

Parte 4 — Aprendendo Props

No tutorial do jogo da velha, o React passa informações para os quadrados usando props.

Vamos fazer o mesmo.

```
function Row({ word }) {
  return (
    <div>
      {word.split("").map((letter, index) => (
        <span key={index}>{letter}</span>
      ))}
    </div>
  );
}
```

Uso:

```
<Row word="CASAS" />
```

O resultado:

C A S A S

Parte 5 — Aprendendo State

No React, State é a memória do componente. O tutorial do jogo da velha usa `useState` para guardar o estado do tabuleiro.

Importe:

```
import { useState } from "react";
```

Crie um estado:

```
const [guess, setGuess] = useState("");
```

- `guess` → valor atual
 - `setGuess` → altera o valor
-

Parte 6 — Campo de texto

Adicione:

```
<input
  value={guess}
  onChange={ (e) => setGuess (e.target.value) }
/>
```

Sempre que digitar, o React atualiza automaticamente a tela.

Esse é um dos conceitos mais importantes do React.

Parte 7 — Guardando tentativas

Precisamos armazenar várias palavras.

```
const [guesses, setGuesses] = useState([]);
```

Exemplo:

```
["CASAS", "LIVRO"]
```

Botão enviar

```
<button onClick={handleSubmit}>
  Enviar
</button>
```

Função:

```
function handleSubmit() {
  if (guess.length !== 5) return;

  setGuesses([...guesses, guess.toUpperCase()]);
  setGuess("");
}
```

Parte 8 — Exibindo tentativas

Substitua as linhas fixas por:

```
{guesses.map((word, index) => (
  <Row key={index} word={word} />
))}
```

Agora cada palavra enviada aparece no tabuleiro.

Parte 9 — Criando a palavra secreta

No começo:

```
const secretWord = "CASAL";
```

Mais tarde você poderá escolher palavras aleatórias.

Parte 10 — Verificando letras

Crie:

```
function getLetterStatus(letter, index, secretWord) {  
  if (secretWord[index] === letter) {  
    return "correct";  
  }  
  
  if (secretWord.includes(letter)) {  
    return "present";  
  }  
  
  return "absent";  
}
```

Parte 11 — Colorindo células

Modifique Row:

```
function Row({ word, secretWord }) {  
  return (  
    <div className="row">  
      {word.split("").map((letter, index) => {  
        const status =  
          getLetterStatus(  
            letter,  
            index,  
            secretWord  
          );  
        return (  
          <span  
            key={index}
```

```

        className={status}
      >
        {letter}
      </span>
    );
  }}}
</div>
);
}

```

Uso:

```

<Row
  word={word}
  secretWord={secretWord}
/>

```

Parte 12 — CSS

Crie em App.css:

```

.row {
  display: flex;
  gap: 5px;
  margin: 5px;
}

.row span {
  width: 50px;
  height: 50px;

  display: flex;
  align-items: center;
  justify-content: center;

  font-size: 24px;
  font-weight: bold;

  border: 1px solid gray;
}

.correct {
  background: green;
  color: white;
}

.present {
  background: goldenrod;
  color: white;
}

.absent {
  background: gray;
  color: white;
}

```

Parte 13 — Limite de 6 tentativas

No envio:

```
function handleSubmit() {  
  
  if (guesses.length >= 6) {  
    return;  
  }  
  
  if (guess.length !== 5) {  
    return;  
  }  
  
  setGuesses([  
    ...guesses,  
    guess.toUpperCase()  
  ]);  
  
  setGuess("");  
}
```

Parte 14 — Vitória

Depois dos estados:

```
const won =  
  guesses.includes(secretWord);
```

Mostre:

```
{won && (  
  <h2>Você venceu!</h2>  
)}
```

Parte 15 — Derrota

```
const lost =  
  guesses.length === 6 &&  
  !won;
```

Exiba:

```
{lost && (  
  <h2>  
    Você perdeu.  
    A palavra era {secretWord}  
  </h2>  
)}
```

Estrutura final do App

Quando terminar, você terá aprendido exatamente os mesmos conceitos fundamentais apresentados no tutorial do jogo da velha:

1. Componentes (`App`, `Row`)
2. Props (`word`, `secretWord`)
3. State (`guess`, `guesses`)
4. Eventos (`onClick`, `onChange`)
5. Renderização de listas (`map`)
6. Atualização imutável de arrays (`[...]`)
7. Fluxo de dados do componente pai para os filhos

Esses são os pilares do React moderno e aparecem em praticamente qualquer aplicação React profissional. O tutorial do jogo da velha usa esses conceitos para controlar um tabuleiro; no Termo, usamos os mesmos conceitos para controlar palavras, tentativas e feedback visual.

Depois de concluir essa versão básica, os próximos desafios são:

- Escolher uma palavra aleatória de uma lista.
- Mostrar um teclado virtual.
- Bloquear letras já usadas.
- Adicionar animações.
- Salvar estatísticas no navegador (`localStorage`).
- Reproduzir o comportamento do Termo original quase exatamente.